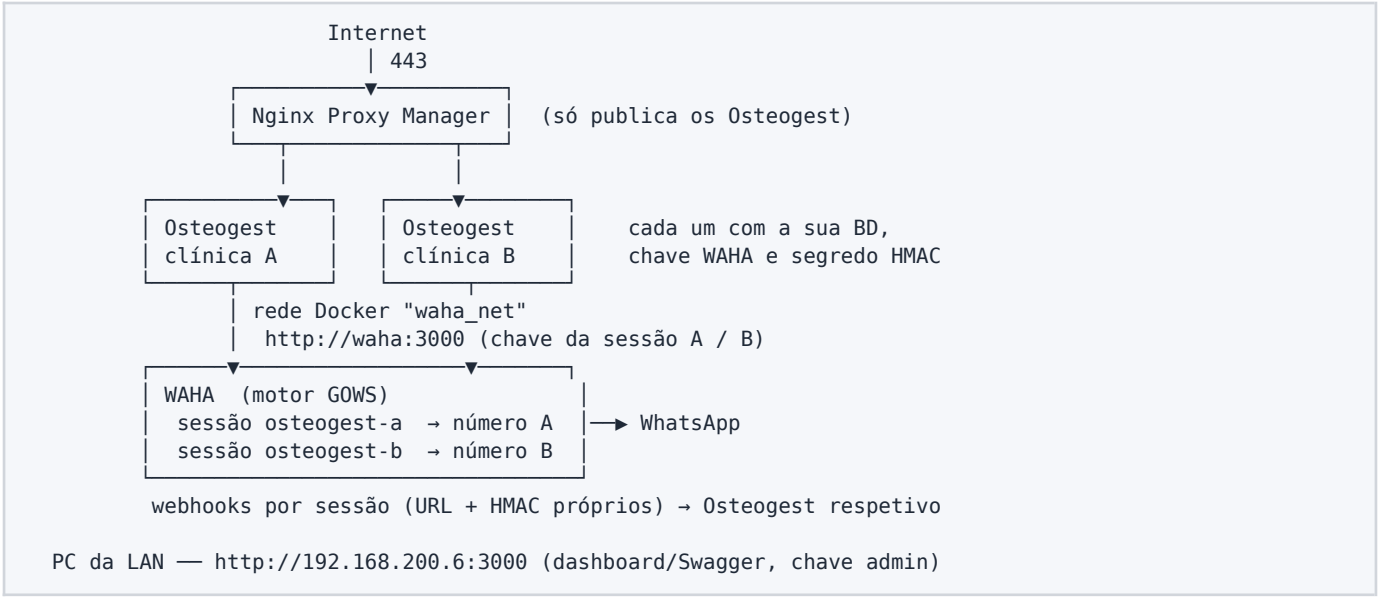
Guia de Deploy — WAHA no Portainer + Nginx Proxy Manager

Projeto: integração WhatsApp do Osteogest (multi-clínica)
Repositório: https://github.com/duquedaniel/WAHA (branch `main`)
Servidores: demo `192.168.200.6` (demo.osteogest.ddx.pt) · produção `vilma.osteogest.ddx.pt`

1. Visão geral

O WAHA (WhatsApp HTTP API) corre como uma stack independente no Portainer e **só é acessível dentro da rede local** do servidor. Cada deployment do Osteogest (uma clínica) tem **a sua própria sessão WhatsApp** (o seu número) e **a sua própria chave de API**, que só dá acesso a essa sessão. Toda a configuração WhatsApp de cada clínica é feita no **backoffice do Osteogest**.



1.1 Modelo de chaves

Chave	Quem a tem	O que permite
Chave admin (`WAHA_API_KEY` da stack)	Só o administrador do WAHA	Criar/apagar sessões, criar/revogar chaves, tudo
Chave da clínica (criada em `/api/keys`)	Backoffice do Osteogest dessa clínica	Apenas a **sua** sessão: ver estado, emparelhar (QR/código), iniciar/parar, configurar webhook, enviar e ler mensagens

Testado na imagem oficial `devlikeapro/waha:gows` — com a chave da clínica A:

Operação	Resultado
Ver/iniciar/configurar a sessão A, obter QR da sessão A	200 (permitido)
Ver, configurar, enviar ou obter QR na sessão B	**403** (bloqueado)
Criar sessões, apagar a sessão A, listar ou criar chaves	**403** (bloqueado)
Chave inválida	**401**

1.2 Princípios de segurança

- A porta do WAHA é publicada **apenas no IP da LAN** (nunca em `0.0.0.0`).
- **Não existe proxy host público** para o WAHA no Nginx Proxy Manager.
- O router **não** faz port-forward da porta 3000.
- A chave admin **nunca** é dada a um Osteogest; cada clínica recebe só a sua chave.
- **Não há webhook global**: cada sessão tem o seu URL de webhook e o seu segredo HMAC-SHA512, configurados pelo próprio Osteogest.

2. Pré-requisitos

- Portainer (Docker standalone) com acesso ao servidor.
- Stack do Osteogest no mesmo servidor (no demo: stack `osteogest-demo`, serviço/container `osteogest`, porta interna `3000`, redes `appnet` e `proxy_net`).
- Nginx Proxy Manager (container `nginx-proxy-manager`, rede `proxy_net`).
- Um número de WhatsApp dedicado por clínica (pode ser WhatsApp Business no telemóvel; o WAHA liga-se como "dispositivo ligado").
- Acesso SSH ao servidor.

3. Gerar os segredos

```
openssl rand -hex 32   # WAHA_API_KEY         (chave admin do WAHA)
openssl rand -hex 32   # WAHA_DASHBOARD_PASSWORD
openssl rand -hex 32   # WHATSAPP_ENCRYPTION_KEY (um por cada deployment do Osteogest)
```

> Guarde-os num gestor de passwords. Use segredos **diferentes** em cada servidor e em cada clínica. As chaves de cada clínica e os segredos HMAC dos webhooks **não** são gerados aqui: a chave da clínica é criada no provisionamento (secção 7) e o HMAC é gerado automaticamente pelo backoffice do Osteogest.

4. Criar a stack do WAHA no Portainer

4.1 Método recomendado — Repository

1. Portainer → **Stacks** → **Add stack**.
2. **Name:** `waha`
3. **Build method:** `Repository`
4. **Repository URL:** `https://github.com/duquedaniel/WAHA`
5. **Repository reference:** `refs/heads/main`
6. **Compose path:** `deploy/portainer/docker-compose.yml`
7. Se o repositório for privado: ativar **Authentication** com o utilizador GitHub + um Personal Access Token (fine-grained, Contents: Read-only apenas neste repositório).
8. Preencher as **Environment variables** (secção 4.3).
9. **Deploy the stack**.

4.2 Método alternativo — Web editor

Cole o conteúdo abaixo (igual a `deploy/portainer/docker-compose.yml`):

```
services:
  waha:
    image: devlikeapro/waha:${WAHA_IMAGE_TAG:-gows}
    container_name: ${WAHA_CONTAINER_NAME:-waha}
    restart: unless-stopped
    dns:
      - 1.1.1.1
      - 8.8.8.8
    # Published only on the server LAN IP, never on 0.0.0.0
    ports:
      - '${WAHA_LAN_IP:?define WAHA_LAN_IP}:${WAHA_LAN_PORT:-3000}:3000/tcp'
    networks:
      - waha_net
    volumes:
      - waha_sessions:/app/.sessions
      - waha_media:/app/.media
    environment:
      TZ: Europe/Lisbon
      WHATSAPP_DEFAULT_ENGINE: GOWS
      WAHA_BASE_URL: http://${WAHA_CONTAINER_NAME:-waha}:3000
      WAHA_LOG_LEVEL: info
      WAHA_PRINT_QR: 'false'
      WHATSAPP_RESTART_ALL_SESSIONS: 'True'

      # Admin (master) key: provisioning only, never given to an Osteogest deployment
      WAHA_API_KEY: ${WAHA_API_KEY:?define WAHA_API_KEY}
      WAHA_DASHBOARD_ENABLED: 'true'
      WAHA_DASHBOARD_USERNAME: ${WAHA_DASHBOARD_USERNAME:-admin}
      WAHA_DASHBOARD_PASSWORD: ${WAHA_DASHBOARD_PASSWORD:?define WAHA_DASHBOARD_PASSWORD}
      WHATSAPP_SWAGGER_ENABLED: 'true'
      WHATSAPP_SWAGGER_USERNAME: ${WAHA_DASHBOARD_USERNAME:-admin}
      WHATSAPP_SWAGGER_PASSWORD: ${WAHA_DASHBOARD_PASSWORD:?define WAHA_DASHBOARD_PASSWORD}

      # No global webhook: each clinic session gets its own webhook URL + HMAC key
      WAHA_MEDIA_STORAGE: LOCAL
      WHATSAPP_FILES_FOLDER: /app/.media
    healthcheck:
      test: ['CMD', 'curl', '-fsS', 'http://localhost:3000/ping']
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 30s
    logging:
      driver: json-file
      options:
        max-size: 50m
        max-file: '5'

networks:
  # Shared with the Osteogest stacks (declare it there as external)
  waha_net:
    name: ${WAHA_NETWORK:-waha_net}

volumes:
  waha_sessions:
  waha_media:
```

4.3 Variáveis de ambiente da stack

```
WAHA_LAN_IP=192.168.200.6
WAHA_LAN_PORT=3000
WAHA_IMAGE_TAG=gows
WAHA_CONTAINER_NAME=waha
WAHA_NETWORK=waha_net
WAHA_API_KEY=<chave admin>
WAHA_DASHBOARD_USERNAME=admin
WAHA_DASHBOARD_PASSWORD=<password dashboard>
```

Variável	Descrição	Demo	Produção
`WAHA_LAN_IP`	IP da LAN onde a porta é publicada	`192.168.200.6`	IP LAN do servidor vilma
`WAHA_LAN_PORT`	Porta no host	`3000`	`3000`
`WAHA_IMAGE_TAG`	Motor/arquitetura da imagem	`gows`	`gows` (ou versão fixa)
`WAHA_CONTAINER_NAME`	Nome do container (hostname na rede interna)	`waha`	`waha`
`WAHA_NETWORK`	Rede partilhada com os Osteogest	`waha_net`	`waha_net`
`WAHA_API_KEY`	**Chave admin** (só para provisionar)	segredo	segredo novo
`WAHA_DASHBOARD_PASSWORD`	Password do dashboard e Swagger	segredo	segredo novo

Imagens disponíveis (`WAHA_IMAGE_TAG`): `gows` (recomendado, sem browser), `noweb` (sem browser), `latest` (WEBJS com Chromium, mais pesado); em ARM: `gows-arm`, `noweb-arm`, `arm`. Em produção pode fixar uma versão (ex.: `gows-<versão>`).

5. Ligar o Osteogest à rede `waha_net`

A stack do WAHA cria a rede `waha_net`. **Faça primeiro o deploy do WAHA** e depois edite a stack do Osteogest (no demo: `osteogest-demo`). Acrescente a rede externa **apenas ao serviço da aplicação** (nunca ao MySQL):

```
services:
  osteogest:
    # ... configuração existente ...
    environment:
      # ... variáveis existentes ...
      WHATSAPP_MODULE_ENABLED: "true"
      WHATSAPP_ENCRYPTION_KEY: ${WHATSAPP_ENCRYPTION_KEY}
    networks:
      - appnet          # existente (MySQL)
      - proxy_net       # existente (NPM)
      - waha_net        # novo

networks:
  # ... manter as definições existentes de appnet e proxy_net ...
  waha_net:
    external: true
```

- **Não remova** `appnet` nem `proxy_net`, senão o Osteogest perde o acesso ao MySQL e ao NPM.
- Adicione `WHATSAPP_ENCRYPTION_KEY` às Environment variables da stack do Osteogest. É a chave com que o Osteogest cifra, na sua base de dados, a chave WAHA da clínica e o segredo HMAC. **Se a perder, terá de voltar a introduzir a chave da clínica no backoffice.**
- **Update the stack**.

Nenhuma chave WAHA vai para variáveis de ambiente do Osteogest: tudo é configurado no backoffice (secção 8).

6. Validar a instalação

6.1 No servidor (SSH)

```
ss -ltnp | grep 3000
# esperado: 192.168.200.6:3000  (NÃO deve aparecer 0.0.0.0:3000 nem [::]:3000)

docker ps --filter name=waha --format '{{.Names}}  {{.Status}}'
# esperado: waha  Up ... (healthy)

docker logs --tail 50 waha
```

6.2 A partir de um PC da rede local

- `http://192.168.200.6:3000/ping` → `{"message":"pong"}`
- `http://192.168.200.6:3000/dashboard` → dashboard (admin + password)
- `http://192.168.200.6:3000/` → Swagger (documentação e consola da API)

6.3 A partir do container do Osteogest (rede interna)

```
docker exec -it osteogest node -e "fetch('http://waha:3000/ping').then(r=>r.text()).then(console.log)"
# esperado: {"message":"pong"}
```

6.4 A partir da internet (telemóvel em 4G)

- `http://<IP público>:3000/ping` → **tem de falhar** (timeout).
- Nenhum domínio público deve abrir o WAHA.

7. Provisionar uma clínica (sessão + chave)

Feito **uma vez por clínica** pelo administrador do WAHA, com a chave admin. O resultado (nome da sessão + chave da clínica) é introduzido no backoffice do Osteogest dessa clínica.

Convenção de nomes da sessão: `osteogest-<clínica>` (letras, números, `-` ou `_`), ex.: `osteogest-demo`, `osteogest-vilma`.

7.1 Com o script (recomendado)

No servidor, a partir do repositório:

```
cd ~/projetos_WAHA/deploy/portainer
./provision-clinic.sh create osteogest-demo http://192.168.200.6:3000
# pede a chave admin (não fica no histórico da shell)
```

O script:

1. Cria a sessão `osteogest-demo` (parada, com metadata `app=osteogest`).
2. Cria uma chave **restrita a essa sessão** com as permissões `read`, `send`, `control`, `setting` (sem `delete` nem `app`).
3. Mostra o **id** e a **key** da chave — guarde-os num gestor de passwords.

Outros comandos:

```
./provision-clinic.sh list http://192.168.200.6:3000               # sessões e chaves
./provision-clinic.sh revoke <key-id> http://192.168.200.6:3000    # revogar uma chave
```

7.2 Pelo Swagger (alternativa, sem SSH)

1. Abrir `http://192.168.200.6:3000/` (utilizador/password do dashboard).
2. **Authorize** → colar a chave admin.
3. **Sessions → POST /api/sessions** com:

```
{ "name": "osteogest-demo", "start": false,
  "config": { "metadata": { "app": "osteogest", "clinic": "osteogest-demo" } } }
```

4. **Api Keys → POST /api/keys** com:

```
{ "isAdmin": false, "isActive": true, "session": "osteogest-demo",
  "actions": { "read": true, "send": true, "control": true,
               "setting": true, "app": false, "delete": false } }
```

5. Copiar o campo `key` da resposta.

7.3 Rodar ou revogar a chave de uma clínica

1. Criar uma nova chave para a mesma sessão (7.1 ou 7.2, passo 4).
2. Colar a nova chave no backoffice do Osteogest e **Testar ligação**.
3. Revogar a chave antiga: `./provision-clinic.sh revoke <key-id-antigo> ...` ou **DELETE /api/keys/{id}** no Swagger.

Para desativar temporariamente sem apagar: **PUT /api/keys/{id}** com `"isActive": false`.

7.4 Remover uma clínica

Com a chave admin: **DELETE /api/sessions/{nome}** (termina a sessão WhatsApp e apaga os dados) e revogar a respetiva chave.

8. Configurar no backoffice do Osteogest

No Osteogest da clínica → **Configurações → Integrações → WhatsApp** (apenas administradores):

Campo	Valor (demo)
URL do WAHA (interno)	`http://waha:3000`
Nome da sessão	`osteogest-demo`
Chave de API da clínica	a `key` do provisionamento
Número WhatsApp da clínica	`351XXXXXXXXX`
URL de callback (webhook, interno)	`http://osteogest:3000/api/webhooks/waha`

Depois:

1. **Testar ligação** — deve mostrar a sessão (estado `STOPPED` na primeira vez).
2. **Aplicar configuração de webhooks** — o Osteogest gera um segredo HMAC próprio e regista o URL de callback na sua sessão WAHA.
3. **Ligar número** — QR code ou código de emparelhamento; no telemóvel da clínica: WhatsApp → **Definições → Dispositivos ligados → Ligar um dispositivo**.
4. O estado passa a **WORKING** e o envio de mensagens fica disponível.

Várias clínicas no mesmo servidor: o URL de callback usa o nome do container de cada Osteogest (ex.: `http://osteogest-vilma:3000/api/webhooks/waha`), por isso os nomes dos containers têm de ser únicos.

9. Configuração do Nginx Proxy Manager

9.1 WAHA — recomendação: nenhum proxy host

A forma mais segura é **não criar** qualquer proxy host para o WAHA. O acesso administrativo faz-se diretamente por `http://192.168.200.6:3000` na LAN e os Osteogest usam `http://waha:3000` pela rede Docker.

9.2 WAHA — opcional: proxy host restrito à LAN (com HTTPS)

Só se quiser um endereço com nome e HTTPS dentro da LAN (ex.: `waha.lan.ddx.pt`).

Passo 1 — Access List (NPM → Access Lists → Add Access List):

Campo	Valor
Name	`LAN-only`
Satisfy Any	desligado
Pass Auth to Host	desligado
Authorization	(vazio — o WAHA já tem autenticação própria)
Access → allow	`192.168.200.0/24`
Access → deny	`all`

Passo 2 — Proxy Host (NPM → Hosts → Proxy Hosts → Add Proxy Host):

Separador	Campo	Valor
Details	Domain Names	`waha.lan.ddx.pt`
Details	Scheme	`http`
Details	Forward Hostname / IP	`192.168.200.6`
Details	Forward Port	`3000`
Details	Cache Assets	desligado
Details	Block Common Exploits	ligado
Details	Websockets Support	**ligado** (necessário para `/ws`)
Details	Access List	**`LAN-only`**
SSL	SSL Certificate	Let's Encrypt com **DNS Challenge** (o HTTP challenge exige acesso público)
SSL	Force SSL / HTTP/2	ligado

Advanced (Custom Nginx Configuration):

```
client_max_body_size 50m;
proxy_read_timeout 300s;
proxy_send_timeout 300s;
```

DNS: `waha.lan.ddx.pt` → `192.168.200.6` (IP privado). Mesmo que alguém de fora resolva o nome, o IP privado não é alcançável da internet e a Access List bloqueia outras origens.

> Atenção: a Access List baseia-se no IP de origem. Se o NPM estiver atrás de outro proxy (ex.: Cloudflare), a restrição deixa de ser fiável — nesse caso use apenas a opção 9.1.

9.3 Osteogest — ajustes no proxy host existente

Nos proxy hosts `demo.osteogest.ddx.pt` e `vilma.osteogest.ddx.pt` (e futuros), separador **Advanced**:

```
# O webhook do WAHA chega pela rede Docker interna; bloquear o acesso público
location ^~ /api/webhooks/waha {
    return 404;
}

# Envio de PDFs/imagens aos pacientes
client_max_body_size 25m;
proxy_read_timeout 120s;
```

Manter Websockets Support e Block Common Exploits ligados.

10. Firewall (opcional, reforço)

O Docker **ignora o UFW** para portas publicadas. O bind ao IP da LAN já impede o acesso externo; para restringir também dentro da LAN:

```
sudo iptables -I DOCKER-USER -p tcp --dport 3000 ! -s 192.168.200.0/24 -j DROP
sudo apt install -y iptables-persistent
sudo netfilter-persistent save
```

Para reverter: `sudo iptables -D DOCKER-USER -p tcp --dport 3000 ! -s 192.168.200.0/24 -j DROP`.

11. Várias clínicas: modelos de isolamento

	Modelo A — um WAHA por servidor (recomendado)	Modelo B — um WAHA por clínica
Isolamento de dados WhatsApp	Por sessão + chave restrita (403 entre clínicas)	Total (processos e volumes separados)
Recursos	Um container para todas	Um container por clínica
Provisionamento	`provision-clinic.sh create` por clínica	Nova stack por clínica + provisionamento
Quando usar	Clínicas no mesmo servidor, geridas por si	Requisitos contratuais de isolamento total

Modelo B no mesmo servidor: criar outra stack (ex.: `waha-vilma`) com o mesmo compose e variáveis diferentes: `WAHA_CONTAINER_NAME=waha-vilma`, `WAHA_NETWORK=waha_net_vilma`, `WAHA_LAN_PORT=3001`, chave admin própria. O Osteogest dessa clínica junta-se a `waha_net_vilma` e usa `http://waha-vilma:3000`.

Nota (Modelo A): todos os Osteogest ligados a `waha_net` conseguem ver-se nessa rede. Ligue a `waha_net` apenas o container da aplicação (nunca a base de dados); cada Osteogest continua protegido pela sua autenticação e cada webhook pela sua assinatura HMAC.

12. Produção (vilma.osteogest.ddx.pt)

- ☐ Descobrir o IP LAN do servidor (`ip -4 addr`) e a sub-rede.
- ☐ Gerar **novos** segredos (secção 3).
- ☐ Criar a stack `waha` com `WAHA_LAN_IP` = IP LAN de produção.
- ☐ Fixar a versão da imagem (`WAHA_IMAGE_TAG=gows-<versão>`).
- ☐ Ligar a stack do Osteogest de produção à rede `waha_net` + `WHATSAPP_ENCRYPTION_KEY`.
- ☐ Provisionar a clínica (`osteogest-vilma`) — secção 7.

☐ Configurar no backoffice do Osteogest — secção 8.
☐ Aplicar os ajustes do NPM (9.3).
☐ Validar (secção 6), incluindo o teste a partir de 4G.
☐ Configurar backups (secção 13).

13. Manutenção

13.1 Atualizar o WAHA

Portainer → Stacks → `waha` → **Pull and redeploy** (Repository) ou **Update the stack** com Re-pull image (Web editor). Sessões e chaves mantêm-se no volume `waha_sessions`.

13.2 Backups

O volume de sessões (`waha_waha_sessions` para a stack `waha`) contém as sessões WhatsApp **e as chaves de API** das clínicas.

```
# backup
docker run --rm -v waha_waha_sessions:/data -v "$PWD":/backup alpine \
  tar czf /backup/waha_sessions_$(date +%F).tgz -C /data .

# restauro (com a stack parada)
docker run --rm -v waha_waha_sessions:/data -v "$PWD":/backup alpine \
  sh -c "rm -rf /data/* && tar xzf /backup/waha_sessions_AAAA-MM-DD.tgz -C /data"
```

> Estes ficheiros permitem enviar mensagens em nome dos números das clínicas. Guarde os backups cifrados e com acesso restrito.

13.3 Rodar a chave admin

1. Gerar novo valor, atualizar `WAHA_API_KEY` na stack `waha`, Update the stack.
2. As chaves das clínicas **não mudam** (estão guardadas no WAHA) — nenhum Osteogest precisa de ser alterado.

14. Resolução de problemas

Sintoma	Causa provável	Ação
`401 Unauthorized`	Chave errada, revogada ou inativa	Confirmar a chave no backoffice; `./provision-clinic.sh list`
`403 Forbidden`	Chave de outra sessão ou operação de admin	Confirmar o nome da sessão no backoffice
`404` na sessão	Sessão não provisionada	Provisionar (secção 7)
Osteogest não resolve `waha`	Osteogest fora da rede `waha_net`	Rever secção 5
Stack não arranca: "define WAHA_..."	Variável obrigatória vazia	Preencher as Environment variables
Sessão em `SCAN_QR_CODE` após reinício	Número desligado no telemóvel / volume perdido	Voltar a emparelhar no backoffice
Sessão `FAILED`	Ligação WhatsApp rejeitada	Reiniciar no backoffice; ver `docker logs waha`
Webhooks não chegam	Callback URL errado ou não aplicado	Backoffice → Aplicar configuração de webhooks; `docker logs waha |`

Sintoma	Causa provável	Ação
		`grep -i webhook`
Webhook 401 no Osteogest	HMAC alterado sem reaplicar	Backoffice → Aplicar configuração de webhooks
Container `unhealthy`	WAHA não responde em `/ping`	Ver logs; confirmar RAM/CPU
Erro 463 ao enviar	Restrição temporária da conta a novos contactos	Reduzir envios a números novos; aguardar

15. Boas práticas de utilização

- Enviar apenas a pacientes com **consentimento** para contacto por WhatsApp.
- Evitar envios em massa e mensagens a números que nunca contactaram a clínica (risco de bloqueio do número).
- Espaçar envios (alguns segundos entre mensagens).
- **Não** incluir diagnósticos, notas clínicas ou outros dados de saúde nas mensagens (RGPD).
- Monitorizar o estado da sessão (o Osteogest alerta quando deixa de estar `WORKING`).